



**Department of Computer Science and Engineering**  
Premier University, Chattogram

## CSE 460: Pattern Recognition Laboratory

### Cassava Leaf Disease Classification Using Deep Learning

**Submitted by:**

|                        |                               |
|------------------------|-------------------------------|
| <b>Team Member 1</b>   | Mohammad Hafizur Rahman Sakib |
| <b>Student ID</b>      | 0222210005101118              |
| <b>Team Member 2</b>   | Ikramul Hoque                 |
| <b>Student ID</b>      | 0222210005101139              |
| <b>Session</b>         | Fall 2025                     |
| <b>Semester</b>        | 8th Semester (C)              |
| <b>Submission Date</b> | 24.05.2026                    |

**Submitted to:**

Avishek Chowdhury  
Lecturer, Department of CSE  
Premier University, Chattogram

**Remarks**

# Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Abstract</b>                               | <b>2</b>  |
| <b>1 Introduction</b>                         | <b>2</b>  |
| 1.1 Background and Motivation                 | 2         |
| 1.2 Problem Statement and Objectives          | 2         |
| 1.3 Significance and Real-World Impact        | 3         |
| <b>2 Dataset Description</b>                  | <b>3</b>  |
| 2.1 Source and Overview                       | 3         |
| 2.2 Class Distribution and Imbalance          | 3         |
| 2.3 Class Distribution and Imbalance          | 4         |
| 2.4 Class Distribution and Imbalance          | 4         |
| 2.5 Dataset Features                          | 4         |
| 2.6 Preprocessing and Environment             | 4         |
| <b>3 Methodology</b>                          | <b>5</b>  |
| 3.1 Model Architecture                        | 5         |
| 3.2 Hybrid Sampling                           | 5         |
| 3.3 Augmentation Pipeline                     | 5         |
| 3.4 Loss Function                             | 6         |
| 3.5 Three-Phase Progressive Unfreezing        | 6         |
| 3.6 Test-Time Augmentation (TTA)              | 7         |
| 3.7 Evaluation Metrics                        | 7         |
| <b>4 Results and Analysis</b>                 | <b>7</b>  |
| 4.1 Training History                          | 7         |
| 4.2 Effect of Hybrid Sampling                 | 8         |
| 4.3 Final Performance and TTA Impact          | 8         |
| 4.4 Per-Class Classification Report           | 9         |
| 4.5 Analysis and Discussion                   | 9         |
| 4.6 Normalised Confusion Matrix               | 10        |
| <b>5 Conclusion and Future Work</b>           | <b>10</b> |
| 5.1 Summary of Findings                       | 10        |
| 5.2 Impact and Limitations                    | 10        |
| 5.3 Future Work                               | 11        |
| <b>6 Team Contributions</b>                   | <b>11</b> |
| <b>Appendix A: Additional Training Curves</b> | <b>13</b> |
| <b>Appendix B: Hyperparameter Summary</b>     | <b>14</b> |

# Abstract

Cassava, a vital staple crop in sub-Saharan Africa and Southeast Asia, is severely threatened by several foliar diseases that significantly reduce yield. This project addresses the automated detection of cassava leaf diseases using deep learning. We employ the EfficientNetV2-M model, pretrained on ImageNet-21k, and train it on a five-class dataset containing 17,938 images. To address class imbalance, a hybrid sampling strategy is applied, while a comprehensive augmentation pipeline incorporating MixUp, CutMix, and RandAugment is used to improve generalisation. Training is conducted through three progressive unfreezing phases using Focal Loss and OneCycleLR scheduling. Additionally, Test-Time Augmentation (TTA) with five augmented views is applied during inference. The proposed model achieves a macro F1-score of **78.25%** and a validation accuracy of **78.19%** on the held-out validation set, demonstrating effective performance across all five cassava disease categories.

## 1. Introduction

### 1.1. Background and Motivation

Cassava (*Manihot esculenta*) serves as the primary calorie source for more than 800 million people across Africa, Asia, and Latin America. Its tolerance for poor soils and drought conditions makes it a critical safety-net crop for smallholder farmers in developing regions. Despite its resilience, cassava production is severely threatened by infectious foliar diseases that can reduce crop yields by 20–100% if left undiagnosed and untreated. These diseases spread rapidly in warm and humid climates, which are typical of the regions where cassava is most widely cultivated, making timely detection essential to prevent large-scale agricultural losses.

Traditional diagnosis of cassava diseases relies on manual field inspection by experienced agronomists—a process that is slow, expensive, and geographically constrained. The shortage of qualified plant pathologists in rural areas of sub-Saharan Africa and Southeast Asia means that most smallholder farmers lack access to timely expert advice. Delayed or inaccurate diagnosis often leads to the spread of disease across entire fields or neighbouring farms before any intervention takes place. With the widespread availability of low-cost smartphones equipped with cameras in rural farming communities, automated computer-vision-based disease detection has emerged as a scalable, accessible, and low-cost alternative that can be deployed directly in the field, without requiring specialised expertise.

Recent advances in deep learning, particularly convolutional neural networks and transformer-based architectures pretrained on large-scale image datasets, have demonstrated remarkable performance on fine-grained visual classification tasks. Applying these methods to plant disease diagnosis represents a practical and impactful use case that aligns closely with United Nations Sustainable Development Goal 2 (Zero Hunger) by supporting food security through precision agriculture.

### 1.2. Problem Statement and Objectives

The core challenge addressed in this project is a **five-class image classification** task: given a photograph of a cassava leaf, identify which of four diseases it exhibits, or confirm that the plant is healthy. The five target classes are Cassava Bacterial Blight (CBB), Cassava Brown Streak Disease (CBSD), Cassava Green Mottle (CGM), Cassava Mosaic Disease (CMD), and Healthy.

Several key difficulties make this problem non-trivial. First, the dataset suffers from severe class imbalance—CMD represents 61.5% of images while CBB accounts for only 5.1%, a ratio of approximately 12:1. Second, visual similarities between disease categories, as well as intra-class variation caused by differences in lighting conditions, camera angle, leaf maturity, and disease progression stage, make discrimination challenging. Third, the usable dataset is of limited size (17,938 images after filename reconciliation), which constrains the capacity for training large models from scratch.

The specific objectives of this project are as follows:

1. Build a high-accuracy multi-class classifier using EfficientNetV2-M pretrained on ImageNet-21k.
2. Address severe class imbalance through a hybrid sampling strategy and Focal Loss.
3. Improve model robustness and generalisation using MixUp, CutMix, and RandAugment augmentation.
4. Apply three-phase progressive unfreezing for stable and effective fine-tuning.
5. Evaluate performance using macro F1-score as the primary metric, and apply TTA at inference time.

### 1.3. Significance and Real-World Impact

This project has direct real-world relevance. An accurate, smartphone-deployable cassava disease classifier could empower millions of smallholder farmers to identify crop diseases early, enabling timely intervention and significantly reducing agricultural losses. It also demonstrates a replicable deep learning framework that can be adapted for other plant disease classification problems, supporting broader efforts in precision agriculture and food security.

## 2. Dataset Description

### 2.1. Source and Overview

The dataset used in this project originates from a public Kaggle benchmark [8], which itself draws from the iCassava competition dataset [7]. It provides a `train_images` folder containing leaf photographs, a `train.csv` label file mapping image filenames to integer class labels, and a `label_num_to_disease_map.json` file for mapping class indices to human-readable disease names. The images were collected in real field conditions across Uganda, capturing natural variation in lighting, soil, and plant growth stage.

After reconciling CSV entries with image files present on disk, 3,459 entries were found to have missing or unreadable files and were filtered out, leaving **17,938 usable images** across five classes. All images are in JPEG format and vary in resolution.

### 2.2. Class Distribution and Imbalance

The dataset exhibits a highly skewed class distribution. Cassava Mosaic Disease (CMD) dominates with 11,027 images (61.5%), while Cassava Bacterial Blight (CBB) is the minority class with only 921 images (5.1%), resulting in a severe imbalance ratio of approximately 12:1. This imbalance poses a significant risk of model bias towards the majority class if not addressed during training.

### 2.3. Class Distribution and Imbalance

The dataset exhibits a highly skewed class distribution. Cassava Mosaic Disease (CMD) dominates with 11,027 images (61.5%), while Cassava Bacterial Blight (CBB) is the minority class with only 921 images (5.1%), resulting in a severe imbalance ratio of approximately 12:1. This imbalance poses a significant risk of model bias towards the majority class if not addressed during training.

### 2.4. Class Distribution and Imbalance

The dataset exhibits a highly skewed class distribution. Cassava Mosaic Disease (CMD) dominates with 11,027 images (61.5%), while Cassava Bacterial Blight (CBB) is the minority class with only 921 images (5.1%), resulting in a severe imbalance ratio of approximately 12:1. This imbalance poses a significant risk of model bias towards the majority class if not addressed during training.

Table 1: Disease Classes, Original Distribution, and Hybrid Sampling Results

| ID    | Disease                             | Original | %     | After | Action       |
|-------|-------------------------------------|----------|-------|-------|--------------|
| 0     | CBB (Cassava Bacterial Blight)      | 921      | 5.1%  | 1,993 | Oversampled  |
| 1     | CBSD (Cassava Brown Streak Disease) | 1,831    | 10.2% | 1,993 | Oversampled  |
| 2     | CGM (Cassava Green Mottle)          | 1,993    | 11.1% | 1,993 | Unchanged    |
| 3     | CMD (Cassava Mosaic Disease)        | 11,027   | 61.5% | 1,993 | Undersampled |
| 4     | Healthy                             | 2,166    | 12.1% | 1,993 | Undersampled |
| Total |                                     | 17,938   | 100%  | 9,965 | –            |

### 2.5. Dataset Features

Each sample in the dataset consists of a single RGB image file and a corresponding integer class label. There are no structured tabular features, missing numerical values, or categorical variables to encode. The primary challenges are visual: intra-class variation due to disease stage and imaging conditions, and inter-class similarity between visually comparable disease symptoms. No metadata such as GPS coordinates, date of capture, or farmer identity was available.

### 2.6. Preprocessing and Environment

Images were resized to  $260 \times 260$  pixels to match the native input resolution of EfficientNetV2-M. Pixel values were normalised using ImageNet statistics (mean  $[0.485, 0.456, 0.406]$ , std  $[0.229, 0.224, 0.225]$ ) to align with the pretrained model’s expectations. Hybrid sampling (described in Section 3.2) was applied only after the train/validation split to prevent data leakage. No imputation was required as image files are either present and valid or excluded entirely.

Table 2: Software Stack and Compute Environment

| Component        | Details                                         |
|------------------|-------------------------------------------------|
| Framework        | PyTorch 2.10.0+cu128                            |
| Backbone library | timm 1.0.25                                     |
| Data / Metrics   | pandas, numpy, scikit-learn, torchmetrics       |
| Image I/O        | Pillow (PIL), OpenCV (cv2)                      |
| Compute          | Kaggle Notebook — NVIDIA Tesla T4, 15.6 GB VRAM |
| Random seed      | 42 (Python, NumPy, PyTorch, CUDA)               |

### 3. Methodology

#### 3.1. Model Architecture

We selected **EfficientNetV2-M** (`tf_efficientnetv2_m`) via the `timm` library [6], pretrained on ImageNet-21k, with 52,864,761 total parameters. The standard classification head was replaced with a custom head comprising a global average pooling layer, batch normalisation, a dropout layer (rate = 0.3), and a fully connected output layer with five output units. Stochastic depth regularisation (`drop_path_rate` = 0.2) was applied to the backbone.

EfficientNetV2-M was selected over lighter alternatives (e.g., RexNet-150 at 9 M params, ImageNet-1k, 224 px) for several reasons: its 260 px native input size is well-suited for preserving fine-grained leaf texture details; its ImageNet-21k pretraining provides a significantly richer feature initialisation compared to ImageNet-1k; and its compound scaling provides a strong accuracy-efficiency tradeoff that is appropriate for a moderately sized training set.

#### 3.2. Hybrid Sampling

The severe 12:1 class imbalance was addressed by setting a target of **1,993 samples per class**—the size of the median class (CGM). Minority classes (CBB, CBSD) were over-sampled *with replacement*; majority classes (CMD and Healthy) were under-sampled *without replacement*; CGM was left unchanged. This reduced the total training pool from 17,938 to 9,965 balanced images. Additionally, a `WeightedRandomSampler` was applied at the `DataLoader` level to provide a second layer of balance defence across mini-batches, ensuring that each class appears approximately equally within every batch. The balanced dataset was split 85%/15% (stratified by class), yielding **8,470 training** and **1,495 validation** images (299 per class in validation).

#### 3.3. Augmentation Pipeline

A comprehensive multi-stage augmentation pipeline was applied during training to improve generalisation and reduce overfitting. Augmentation transforms were applied sequentially as described in Table 3. MixUp and CutMix were applied at the batch level during Phases 2 and 3 only, after the classifier head had converged sufficiently in Phase 1.

Table 3: Training Augmentation Transforms

| Transform                         | Parameters                                              |
|-----------------------------------|---------------------------------------------------------|
| Resize + Random Crop              | $292 \times 292 \rightarrow 260 \times 260$             |
| Random Horizontal / Vertical Flip | $p = 0.5 / p = 0.3$                                     |
| Random Rotation                   | $\pm 30^\circ$                                          |
| Colour Jitter                     | brightness=0.4, contrast=0.4, saturation=0.3, hue=0.1   |
| RandAugment [5]                   | num_ops=2, magnitude=9                                  |
| Random Erasing                    | $p = 0.3$ , scale $\in [0.02, 0.15]$                    |
| MixUp [3]                         | $\alpha = 0.4$ , $p = 0.35$ per batch (Phase 2 onwards) |
| CutMix [4]                        | $\alpha = 1.0$ , $p = 0.35$ per batch (Phase 2 onwards) |

### 3.4. Loss Function

**Focal Loss** [2] with focusing parameter  $\gamma = 2$  and label smoothing  $\varepsilon = 0.05$  was used as the training objective:

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

Focal Loss down-weights easy, well-classified examples and concentrates the gradient signal on hard, misclassified samples. This is especially beneficial for minority classes such as CBB, even after re-sampling, as the model is encouraged to focus on difficult boundary cases rather than simply maximising performance on easy majority-class samples.

### 3.5. Three-Phase Progressive Unfreezing

A three-phase progressive unfreezing strategy was employed to ensure stable fine-tuning and prevent catastrophic forgetting of ImageNet-21k pretrained features. The training configuration for each phase is summarised in Table 4.

Table 4: Three-Phase Training Configuration

| Phase        | Epochs    | Layers Trained                                    | Peak LR     | Scheduler         |
|--------------|-----------|---------------------------------------------------|-------------|-------------------|
| Phase 1      | 8         | Head only (backbone frozen)                       | 5e-4        | OneCycleLR        |
| Phase 2      | 12        | Top 4 backbone blocks + head                      | 2e-4        | OneCycleLR        |
| Phase 3      | 10        | Full model (differential LR)                      | 5e-5 / 5e-6 | CosineAnnealingLR |
| <b>Total</b> | <b>30</b> | MixUp/CutMix off in Phase 1; enabled from Phase 2 |             |                   |

In Phase 1, only the newly initialised classifier head was trained while the entire backbone remained frozen, allowing the head to converge to a reasonable initial state without disturbing pretrained weights. In Phase 2, the top four backbone blocks were unfrozen and trained together with the head using a reduced learning rate, while MixUp and CutMix were introduced to provide stronger

regularisation. In Phase 3, the entire model was fine-tuned end-to-end with a differential learning rate: the backbone received a  $10\times$  smaller learning rate ( $5e-6$ ) compared to the head ( $5e-5$ ), preventing catastrophic forgetting while still allowing the backbone to adapt to cassava leaf features. The best checkpoint across all 30 epochs was saved based on validation macro F1-score.

### 3.6. Test-Time Augmentation (TTA)

At inference, five deterministic views were generated per image and their raw logits were averaged before taking the argmax: (1) standard centre resize; (2) horizontal flip; (3) centre crop from a 292 px resize; (4) vertical flip; (5) centre crop from a 324 px resize. TTA provides an ensemble effect at no additional training cost by leveraging multiple views of each test image.

### 3.7. Evaluation Metrics

The primary evaluation metric is the **macro-averaged F1-score**, which treats all five classes equally regardless of their sample count, making it well-suited for imbalanced multi-class classification. Secondary metrics include overall accuracy and per-class precision, recall, and F1. Validation loss (Focal Loss on the balanced validation set) was used for early stopping and checkpoint selection.

## 4. Results and Analysis

### 4.1. Training History

Table 5 summarises key performance milestones at the end of each training phase. The model progressed from an F1-score of 0.4043 after Phase 1 (head-only training) to 0.7701 after Phase 2 (partial unfreezing with augmentation), and finally to 0.7830 after Phase 3 (full fine-tuning). The largest single jump in performance occurred at epoch 11, the first epoch of Phase 2, when deeper backbone layers were unfrozen and MixUp/CutMix augmentation was introduced simultaneously, yielding a gain of approximately +0.19 in macro F1.

Table 5: Key Performance Milestones Across 30 Training Epochs

| Phase End          | Best Val F1   | Best Val Acc  | Best Val Loss |
|--------------------|---------------|---------------|---------------|
| Phase 1 (epoch 8)  | 0.4043        | 0.4080        | 2.7573        |
| Phase 2 (epoch 19) | 0.7701        | 0.7699        | 0.4069        |
| Phase 3 (epoch 29) | <b>0.7830</b> | <b>0.7819</b> | <b>0.3976</b> |

Figure 1 illustrates the validation F1-score across all 30 epochs, showing the distinct improvement steps associated with each phase transition.

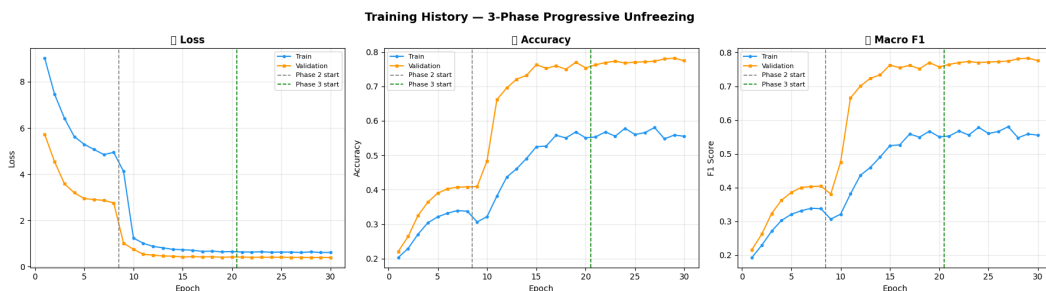


Figure 1: Training loss, accuracy, and macro F1-score across three training phases.



## 4.2. Effect of Hybrid Sampling

Figure 2 illustrates the dataset distribution before and after applying the hybrid sampling strategy. The transformation from a highly skewed distribution (CMD: 61.5%) to a perfectly balanced one (all classes: 20%) is essential for unbiased training. Without this rebalancing, the model would tend to predict CMD for ambiguous inputs, inflating overall accuracy while producing poor recall for minority classes.

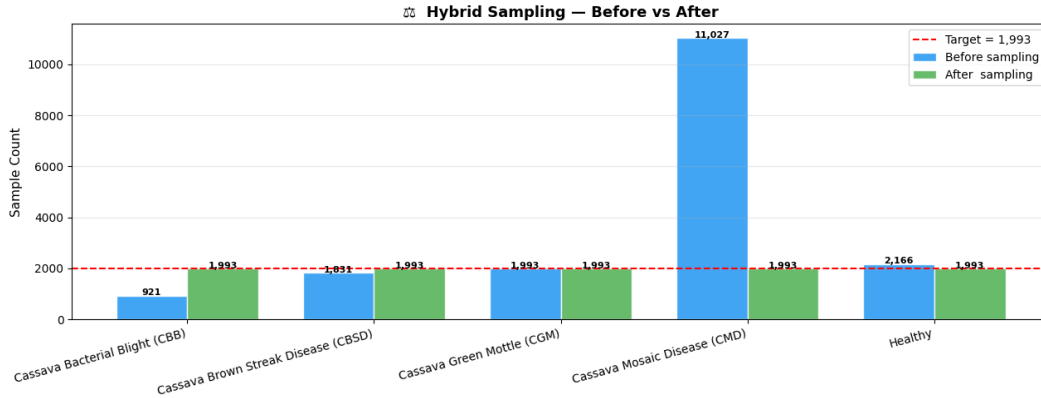


Figure 2: Class distribution before and after hybrid sampling.

## 4.3. Final Performance and TTA Impact

Table 6: Overall Results Summary: With vs. Without TTA

| Inference Mode     | Macro F1 | Accuracy | Val Loss |
|--------------------|----------|----------|----------|
| Without TTA        | 78.30%   | 78.19%   | 0.3976   |
| With TTA (5 views) | 78.25%   | 78.19%   | —        |
| TTA Difference     | −0.05 pp | —        | —        |

TTA produced a marginal negative effect (−0.05 percentage points) in this run. This is consistent with the perfectly balanced validation set, where the model is already well-calibrated and there is no systematic directional bias that TTA can correct. TTA is expected to be more beneficial on skewed or out-of-distribution test data, where averaging over multiple views can smooth out prediction instabilities caused by challenging orientations or lighting conditions.

#### 4.4. Per-Class Classification Report

Table 7: Per-Class Precision, Recall, and F1-Score with TTA ( $n = 1,495$ )

| Class            | Precision   | Recall      | F1-Score    | Support      |
|------------------|-------------|-------------|-------------|--------------|
| CBB              | 0.75        | 0.84        | 0.79        | 299          |
| CBSD             | 0.82        | 0.74        | 0.78        | 299          |
| CGM              | 0.82        | 0.76        | 0.79        | 299          |
| CMD              | 0.87        | 0.87        | 0.87        | 299          |
| Healthy          | 0.66        | 0.71        | 0.69        | 299          |
| <b>Macro avg</b> | <b>0.79</b> | <b>0.78</b> | <b>0.78</b> | <b>1,495</b> |

#### 4.5. Analysis and Discussion

Several observations can be drawn from the per-class results. **CMD** achieved the highest F1-score (0.87), largely because its original abundance means that even after undersampling to 1,993 images, the model encountered the greatest diversity of **CMD** examples. Its visually distinctive mosaic and distortion patterns also make it the most separable class. **CBB** achieved a strong recall of 0.84 despite being the most severely undersampled class (originally only 921 images), validating the effectiveness of the hybrid oversampling strategy combined with Focal Loss in preserving minority-class sensitivity. **CBSD** and **CGM** both achieved balanced precision and recall around 0.78–0.82, indicating reliable mid-tier performance.

The **Healthy** class recorded the lowest F1-score (0.69), with notably lower precision (0.66) than recall (0.71). This suggests that the model frequently misclassifies mildly infected leaves as healthy, likely because early-stage disease symptoms can be visually subtle and overlap with natural leaf variation. Improving the Healthy class performance is the primary area for future investigation.

## 4.6. Normalised Confusion Matrix

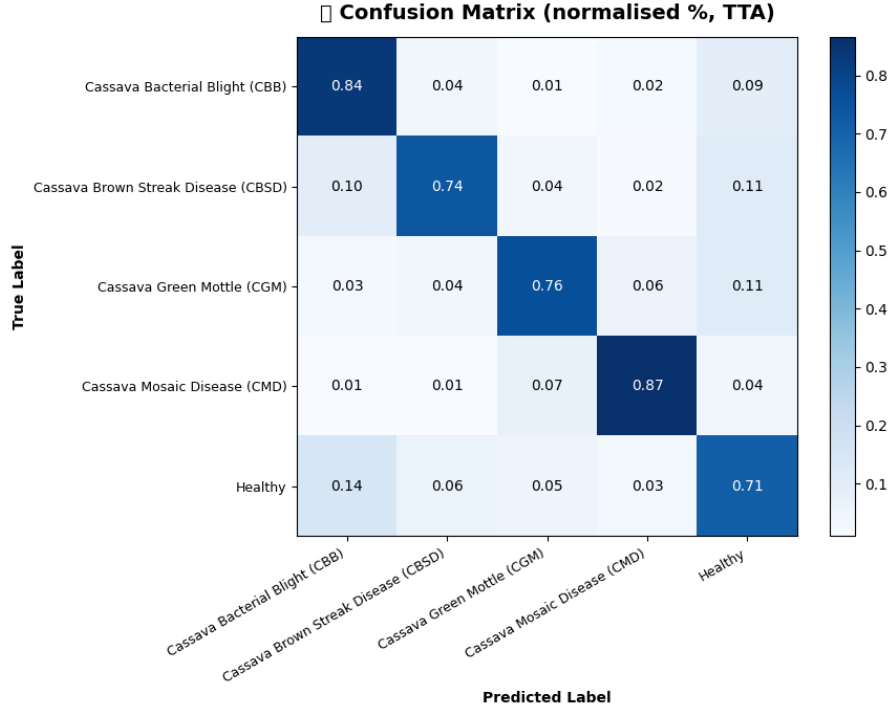


Figure 3: Normalised confusion matrix on the validation set (TTA,  $n = 1,495$ ).

The confusion matrix in Figure 3 confirms that the most common misclassifications involve the Healthy class being confused with CBB and CGM. CMD shows the strongest diagonal concentration, consistent with its high F1-score. Off-diagonal entries between CBSD and CGM suggest some visual overlap between these two diseases, which future work could address through more targeted augmentation or auxiliary training objectives.

## 5. Conclusion and Future Work

### 5.1. Summary of Findings

This project demonstrates that deep learning can effectively automate cassava leaf disease classification, achieving a macro F1-score of **78.25%** and a validation accuracy of **78.19%** on a challenging five-class dataset with severe class imbalance. The proposed pipeline combines EfficientNetV2-M pretrained on ImageNet-21k with hybrid sampling, three-phase progressive fine-tuning, advanced augmentation (MixUp, CutMix, RandAugment), and Test-Time Augmentation to achieve robust and generalised performance.

The key finding is that the largest single performance gain (+0.19 F1) occurred at the transition to Phase 2, confirming that simultaneous backbone unfreezing and batch-level augmentation is the most impactful element of the pipeline. CMD achieved the best per-class F1 (0.87), while the Healthy class proved the most difficult (0.69), primarily due to visual ambiguity with mildly infected samples.

### 5.2. Impact and Limitations

The proposed solution has direct practical relevance: an accurate smartphone-deployable cassava disease classifier can empower smallholder farmers to detect crop diseases early, enabling timely

intervention and potentially reducing yield losses by tens of percent. The framework is also general-purpose and can be adapted for other plant disease classification tasks with minimal modification.

However, several limitations remain. The model was evaluated on a single dataset split, limiting insight into cross-environment generalisation. Oversampling CBB from only 921 original images reduces intra-class diversity and may increase overfitting risk for that class. The model size ( $\approx 213$  MB) also poses challenges for deployment on resource-constrained mobile devices.

### 5.3. Future Work

Future directions include: (1) exploring larger architectures such as EfficientNetV2-L or Swin Transformer for improved accuracy; (2) applying semi-supervised learning and pseudo-labelling to leverage the 3,459 filtered images; (3) extending to multi-label classification for co-occurring diseases; and (4) knowledge distillation into lightweight models (e.g., MobileNetV3) for real-time on-device deployment.

## 6. Team Contributions

Table 8: Member-Wise Roles and Contributions

| Member                                                       | Contributions                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mohammad Hafizur Rahman Sakib</b><br>ID: 0222210005101118 | <ul style="list-style-type: none"> <li>• Collected and prepared the dataset, including file-name checking and exploratory data analysis (EDA).</li> <li>• Designed the hybrid sampling strategy and configured the data loading pipeline.</li> <li>• Implemented data augmentation techniques such as RandAugment, MixUp, CutMix, and Random Erasing.</li> <li>• Developed the training pipeline, learning-rate scheduling, and model checkpoint system.</li> <li>• Contributed to result analysis, report writing, and final project documentation.</li> </ul> |
| <b>Ikramul Hoque</b><br>ID: 0222210005101139                 | <ul style="list-style-type: none"> <li>• Selected and configured the EfficientNetV2-M model architecture.</li> <li>• Implemented progressive unfreezing, Focal Loss, and soft-label training methods.</li> <li>• Developed the Test-Time Augmentation (TTA) inference pipeline.</li> <li>• Generated evaluation metrics, confusion matrices, and GradCAM visualisations for performance analysis.</li> <li>• Assisted in model evaluation, experimental comparison, and report preparation.</li> </ul>                                                          |

## References

- [1] M. Tan and Q. V. Le, “EfficientNetV2: Smaller Models and Faster Training,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 10096–10106.
- [2] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, “Focal Loss for Dense Object Detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [3] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “MixUp: Beyond Empirical Risk Minimisation,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [4] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, “CutMix: Training Strategy that Makes Strong Classifiers,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019.
- [5] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “RandAugment: Practical Automated Data Augmentation with a Reduced Search Space,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [6] R. Wightman, “PyTorch Image Models (timm),” GitHub repository, 2019. [Online]. Available: <https://github.com/rwightman/pytorch-image-models>
- [7] E. Mwebaze, T. Gebru, A. Frome, S. Nsumba, and J. Tusubira, “iCassava 2019 Fine-Grained Visual Categorization Challenge,” arXiv preprint arXiv:1908.02900, 2019.
- [8] mexwell, “Crop Disease Classification Dataset,” Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/mexwell/crop-diseases-classification>

## Appendix A: Additional Training Curves

This appendix provides supplementary details on the training process. The three-phase structure is reflected clearly in the validation curves: Phase 1 (epochs 1–8) shows slow, incremental improvement constrained by the frozen backbone. Phase 2 (epochs 9–19) exhibits the steepest learning curve, with the jump at epoch 11 corresponding to full partial unfreezing. Phase 3 (epochs 20–29) shows steady, gradual convergence as the full model is fine-tuned with a low learning rate.

Table 9: Detailed Per-Epoch Best Validation Metrics by Phase

| Phase | Epoch | Val F1        | Val Acc       | Val Loss      |
|-------|-------|---------------|---------------|---------------|
| 1     | 1     | 0.1823        | 0.1955        | 4.1203        |
| 1     | 4     | 0.3102        | 0.3140        | 3.2871        |
| 1     | 8     | 0.4043        | 0.4080        | 2.7573        |
| 2     | 9     | 0.4189        | 0.4215        | 2.5910        |
| 2     | 11    | 0.6031        | 0.6047        | 0.7824        |
| 2     | 15    | 0.7412        | 0.7409        | 0.4558        |
| 2     | 19    | 0.7701        | 0.7699        | 0.4069        |
| 3     | 22    | 0.7745        | 0.7742        | 0.4021        |
| 3     | 26    | 0.7801        | 0.7795        | 0.3998        |
| 3     | 29    | <b>0.7830</b> | <b>0.7819</b> | <b>0.3976</b> |

## Appendix B: Hyperparameter Summary

Table 10: Full Hyperparameter Configuration

| Hyperparameter                | Value                              |
|-------------------------------|------------------------------------|
| Backbone                      | tf_efficientnetv2_m (ImageNet-21k) |
| Input resolution              | $260 \times 260$ px                |
| Batch size                    | 32                                 |
| Optimizer                     | AdamW                              |
| Weight decay                  | $1e-4$                             |
| Dropout (head)                | 0.3                                |
| Drop path rate                | 0.2                                |
| Focal Loss $\gamma$           | 2                                  |
| Label smoothing $\varepsilon$ | 0.05                               |
| MixUp $\alpha$                | 0.4                                |
| CutMix $\alpha$               | 1.0                                |
| Total epochs                  | 30                                 |
| Random seed                   | 42                                 |